

Lecture 01 — Algorithms and Programs

Algorithmic Thinking · Week 1, Session 1 — Tuesday 22 September 2026,
17:35–18:55

Leopoldo Rodríguez Salazar · Tecnológico de Monterrey

Contents

Overview	1
Day-one logistics — who, how, when	1
Notes	3
Stretch — if the session was easy	5
After the session	5

Objective. Write an algorithm precise enough that a hostile reader executing it literally cannot produce a wrong result.

Overview

Day one. Most of this room has never written a program. That is fine — and it is not what we start with anyway.

We start with the one thing that separates a program that works from a program that nearly works: **precision**. A computer is not a collaborator who fills in what you meant. It is the most literal reader you will ever write for, and it executes exactly what you wrote, including the part you thought was obvious.

So today there is no Python. Today we write instructions in Spanish or English, on paper, and then I execute them adversarially — exactly as written, not as intended — until they break. By the end you will know, physically, why an algorithm needs every step stated, in order, finite, and unambiguous.

That is the whole lesson. One idea.

Day-one logistics — who, how, when

Fifteen minutes, after the square and before the definitions, while the two handouts go round. Slides 4–9 of the deck carry it; the handouts carry the detail. One or two sentences per item — do not read the handout aloud.

Who. Leopoldo Rodríguez Salazar — *Leo*. PhD, aerospace and automotive engineering; Jaguar Land Rover, GE Aviation, Ford; Marie Curie Fellow. Now CEO of Helgen Technologies in Querétaro, building telemetry hardware and the software behind it. *I write algorithms for a living; the code has never been the hard part* — that sentence is

the course. rodriguez.l@tec.mx, Canvas Inbox; asesorías Tuesday 16:30-17:30, Room 4206.

The week, every week.

Tuesday	17:35-18:55, Room 4206	quick exam in the first 20 minutes (from week 2), then the session
Friday	16:30-17:30, Room 4206	laboratory with Luis Castañón (TA)
Friday	17:35-18:55, Room 4206	session
Sunday	23:59, Canvas	homework due, when there is one
Tuesday	16:30-17:30, Room 4206	asesorías, in person

Attendance is taken 17:35-17:40; after 17:40 you are absent. Graded work happens in the room — miss the session and the activity is zero (academia rule, not mine).

Ground rules. Paper first, always. No AI in the room or in the lab; outside, to understand, never to produce — and you will explain your own code out loud on project deliverables. Laptops closed unless we are typing; phones away. Exams: Respondus LockDown Browser, Trinket editor, no Thonny, **no Macs** — if you only have a Mac, say so on Friday. Deadlines are fixed. An integrity breach is a failing grade and goes to the campus committee.

How you are graded. 4 % study the levelling module · 1 % final diagnostic · 12 % quick exams (5) · 10 % laboratories (10) · 4 % in-class activities (4) · 4 % homework (8) · 20 % midterm · 15 % situación problema · 30 % final. Rubrics are published before every exam. Sixty-five percent of the grade is individual, closed exams — anything that gets you through homework without understanding it is a bad trade.

Key dates. Quick exam 1 **Tue 29 Sep** (first 20 minutes, on this week) · final diagnostic **Fri 9 Oct** (lab) · midterm **Fri 23 Oct** · situación problema deliverables **Nov 6 / 13 / 20** (teams of 2), final **Fri 27 Nov** (individual), -5 per missed stage · last day to drop **Fri 13 Nov** · final exam **Tue 1 Dec**.

The levelling module (5 %). An adaptive online module (*Realizeit*, opened from Canvas): ten short lessons — computer memory, arithmetic / relational / logical expressions, algorithms, flowcharts (four lessons), problem solving. **Friday 16:30, in the lab**, a 20-minute diagnostic on the laptop unlocks the lessons and decides which ones each student needs; score 100 and you are done. Then self-paced: lessons 1-5 by Tue 29 Sep, all ten by **Thu 8 Oct**; final diagnostic **Fri 9 Oct** in the lab. Before Friday: bring the laptop, and set the browser language to Spanish (Mexico) or English with Chrome's auto-translate off — otherwise the platform shows , , for decimals and rejects correct answers. Platform questions go to TECservices (TECservices@servicios.tec.mx, WhatsApp 81 1625 5123), not to me. Full requirements: ../ADAPTIVE-MODULE.md.

Notes

What a computer actually is, for our purposes

A machine that executes a sequence of instructions, one at a time, in order, without judgement. It has memory (somewhere to put values), it can do arithmetic, it can compare, and it can choose what to do next based on that comparison. That is the entire vocabulary.

Everything you build this semester is made of those four things. Not more.

Gloss: *memoria* — memory · *instrucción / sentencia* — statement · *ejecutar* — to execute

Algorithm vs. program

	Algorithm	Program
What it is	The thinking	The typing
Written in	Natural language, pseudocode, a flowchart	Python
Read by	A human (and, adversarially, by you)	A machine
Can be wrong by	Being ambiguous, unordered, unending	...all of that, plus syntax

Notice the last row. A program can fail for every reason an algorithm can fail, **and** for reasons of its own. If you skip the algorithm, you are debugging both layers simultaneously, without knowing which one is broken. That is why we separate them, and that separation is the single most useful habit in this course.

The four properties

Finite. It ends. “Keep adding numbers” is not an algorithm. “Keep adding numbers until the user types 0” is. The most common way first-semester code hangs forever is a step that has no stopping condition — we will meet this properly in Week 3, but it starts here.

Ordered. The sequence is part of the meaning. *Break the eggs, then heat the pan* and *heat the pan, then break the eggs* are different algorithms with different results.

Unambiguous. Exactly one reading. “Add the numbers” — which numbers? All of them? The last two? In what order? If two people following your algorithm can get different answers, it is ambiguous, and ambiguity is a bug you have not found yet.

Defined input and output. You know what you are given and what you must produce. This is the step students skip. Before writing a single step, write two lines:

INPUT: the distance in kilometres, the time in hours

OUTPUT: the average speed in km/h

Half the ambiguity dies right there.

Pseudocode — our notation

Pseudocode is structured natural language. There is no official syntax; the convention for this course:

```
ALGORITHM  average_speed
INPUT:      distance (km), time (hours)
OUTPUT:     speed (km/h)
```

```
1. READ distance
2. READ time
3. speed ← distance / time
4. DISPLAY speed
END
```

- $\leftarrow$ is assignment — “*gets the value of*”. Not equality. `speed ← distance / time` is a command, not a claim.
- READ gets a value from the user. DISPLAY shows one.
- Steps are numbered. Numbering is not decoration: it is what makes “ordered” checkable.

Gloss: *asignación* — assignment · *pseudocódigo* — pseudocode · *entrada / salida* — input / output

Writing an algorithm that survives a hostile reader

The working method, which is also the exam method and also the method you will use in your final project:

1. **State input and output.** Two lines, before anything else.
2. **Write the steps, numbered.** One action per step. If a step contains “and”, it is probably two steps.
3. **Attack it.** Read your own algorithm as literally as I read your square. Look specifically for: a step that assumes something unstated; two steps whose order you never decided; a quantity you named two different ways; a case where the input is empty, zero, or negative.
4. **Fix and repeat.**

Step 3 is the one that separates a passing grade from a good engineer. Everyone can do step 2.

Exercise 1 — Price validation (*individual, on paper, 10 min*)

Write an algorithm that checks whether a price entered by the user is valid. A price is valid if it is a positive value.

Then attack your own algorithm and answer, in writing, underneath it:

- What does yours do if the user enters exactly 0?
- What does yours do if the user enters hola?
- Does your algorithm say what “valid” *looks like* as an output — a word, a symbol, nothing?

Nobody’s first version answers all three. That is the point of the exercise.

Exercise 2 — Metres to feet (*pairs, on paper, 15 min*)

Write an algorithm that, given a length in metres, calculates and displays the equivalent in feet. You are given: 1 foot = 12 inches, 1 inch = 2.54 cm, 1 m = 100 cm.

Constraints, and these are the exercise:

- Your algorithm must have **numbered steps** and explicit INPUT: / OUTPUT: lines.
- **Swap with the other pair next to you.** Execute their algorithm literally, on paper, for an input of 3.5 metres. Write down the number you get by following their steps exactly — not the number you know is right.
- If you got a different answer from theirs, find the step where the two of you diverged. That step is ambiguous. Write down which word caused it.

Hand in: your algorithm, your partner-pair's execution of it, and the ambiguous word.

Stretch — if the session was easy

Optional, ungraded, and harder than they look. Bring them to asesorías or leave them in the week's evidence box. No Python needed — none of these is a programming problem.

1. **Seven questions.** I think of a whole number from 1 to 100; you may only ask questions I answer *yes* or *no*. Write the algorithm — numbered steps, no gaps — that always finds the number in **seven questions or fewer**. Then argue, on paper, why six can never be enough. (Count what six yes/no answers can distinguish.)
2. **Euclid, executed.** The oldest algorithm still in daily use: *to find the largest number that divides both a and b, replace the larger by the difference until both are equal*. Write it as INPUT / OUTPUT / steps. Execute it by hand for (48, 18) and for (1071, 462), counting the steps. Then answer the only question that matters for an algorithm: *why must it stop?*
3. **The literal friend.** Write instructions for drawing a five-pointed star with one continuous line. Give them to someone who follows them **literally** and says nothing. Every point where they hesitate or draw something else is a missing line in your algorithm — write those lines. How many did you need?

After the session

Friday is a long day: laboratory 16:30-17:30 in this room, then our session at 17:35.

Bring your laptop, a notebook and a pencil on Friday. The lab opens with a 20-minute prior-knowledge diagnostic that needs the laptop; everything after it is on paper. Also bring today's algorithm — you will be attacking each other's.

Before Friday:

1. **Install Thonny** — see informacion/instalacionThonny.html. Bring a working installation on Friday; we write our first program then.
2. **Install Respondus LockDown Browser** — see [ComoUtilizarRespondusAlumnos.pdf](#). Complete the practice exam by **Monday 28 September, 23:59**. *If you only have a Mac, tell me on Friday — Macs are not permitted for exams and we need to arrange something before your first quick exam.*

3. **Adaptive levelling module** — nothing before Friday. It opens *with* the diagnostic at 16:30 in the lab: the diagnostic decides which of its ten lessons you must study, and you cannot open the lessons until you have taken it.
4. **Read** the two handouts given out today: the course policies and *Using AI in this course*. Friday's session opens with your questions on them.